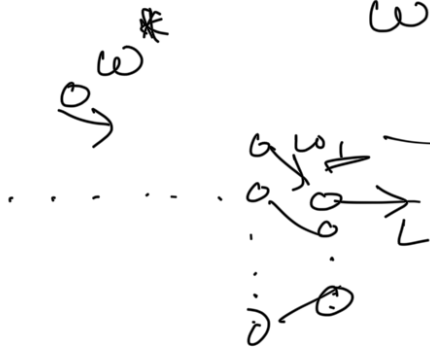


Vanishing Gradients

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$



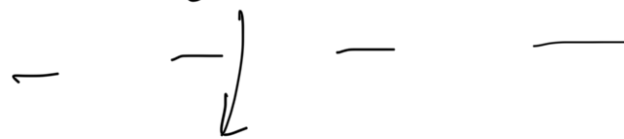
$$L = \sum_{i=1}^M \sum_{j=1}^K I_i \log P(Y_i = j)$$

$L \Rightarrow$ function of output

weights in output layer
are directly related to L

w^* is very far from the output L

$$\frac{\partial L}{\partial w^*} \Rightarrow \text{chain rule}$$



product of gradients

$$\begin{aligned} &L \\ &\leq 1 \\ &\underline{\underline{\leq 1}} \end{aligned}$$

due to repeated multiplication
gradient w.r.t to
weights in input
layers (beginning layers)

is very small.

$$w_{new}^* = w_{old}^* - \underbrace{\eta \frac{\partial L}{\partial w^*}}$$

← Update is low
parameter is not updated

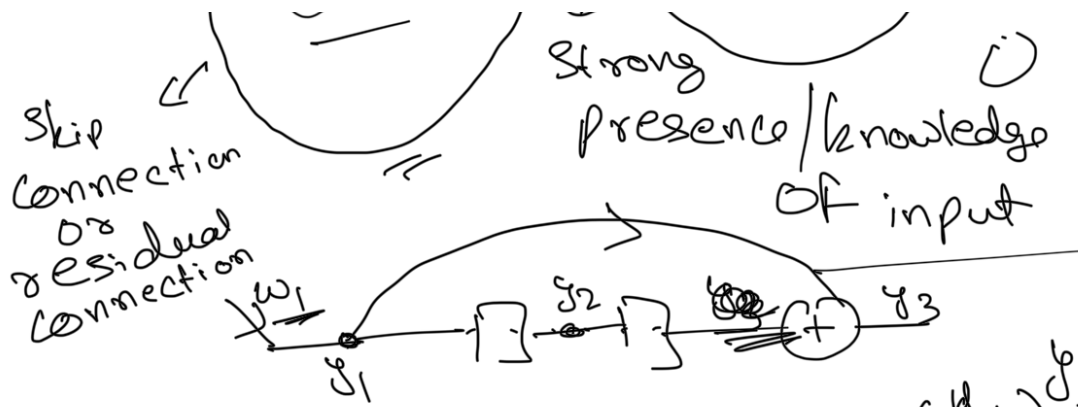
No learning

← Problem of vanishing gradient.

ResNet (for CV applications)
proposed by researchers at microsoft
← Residual networks

LeNet





$$\frac{\partial y_3}{\partial w_1} = \frac{\partial (y_2^{old} + y_1)}{\partial y_2} \times \frac{\partial y_3}{\partial y_1} \times \frac{\partial y_1}{\partial w_1}$$

$y_3 = f(y_2, y_1)$

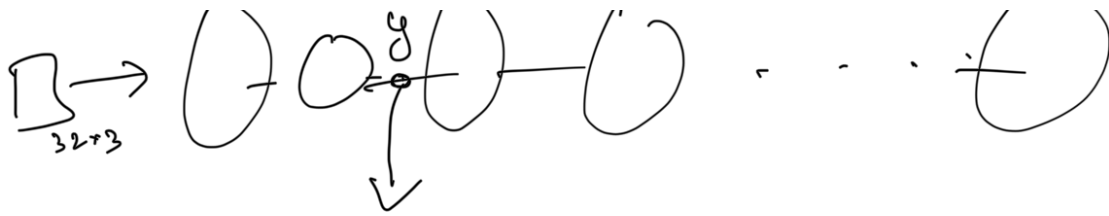
ResNet 50
 # of layer $\Rightarrow$ 220k Parameters

ResNet 50
 50 layers $\Rightarrow$ 2.5M

Batch Normalization

Another processing step
 in deep learning
Architectures

2D array



what is the guarantee

No guarantee

as there is no constraints on outputs of layers,

that y is statistically well-behaved!
 - $\{$ values lying in a specific range $\}$

To make the outputs of layers, $\Rightarrow$ statistically well behaved
 Batch normalization is used.

Every DL model is given

input as batches

$\downarrow$
 in this case $\Rightarrow$ images as batches

$\downarrow$
 let $x_i \Rightarrow$ be one input in the batch

Assume N images in

the batch



$$x_i^* = \alpha \left(\frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \right) + \beta //$$

$$\mu = \frac{1}{L} \sum_{i=1}^L x_i$$

$$\sigma^2 = \frac{1}{L} \sum_{i=1}^L (x_i - \mu)^2$$

$\alpha, \beta \Rightarrow$ trainable parameters (Parameter of the layer)

$\epsilon \Rightarrow$ hyperparameter (defined before training)

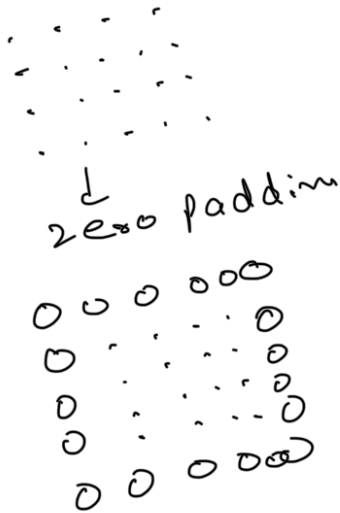
to prevent
division
by
zero

Default
values
for ϵ

Pytorch	Tensorflow
$\epsilon = 10^{-5}$	$\epsilon = 10^{-3}$

out-c $\Rightarrow$ out-channels

Padding
(zero padding)



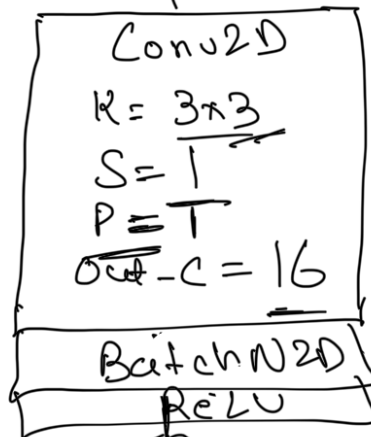
Within
a
single
stage
of
channels
remain
same.

Every
block
consists
of
2 conv layers.

ResNet 20

Input
(32x32x3)

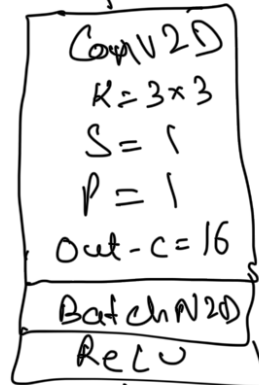
$\Rightarrow$ color images



$\Rightarrow$ Initial layer
(increased #
of
channels to
16)

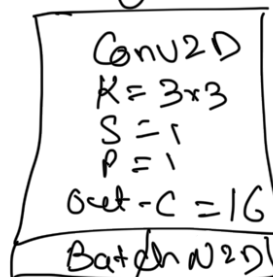
3 stages
(Each stage
has
six
convolutional
layers)

32x32x16



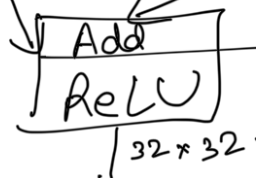
S1-b1-c1

32x32x16

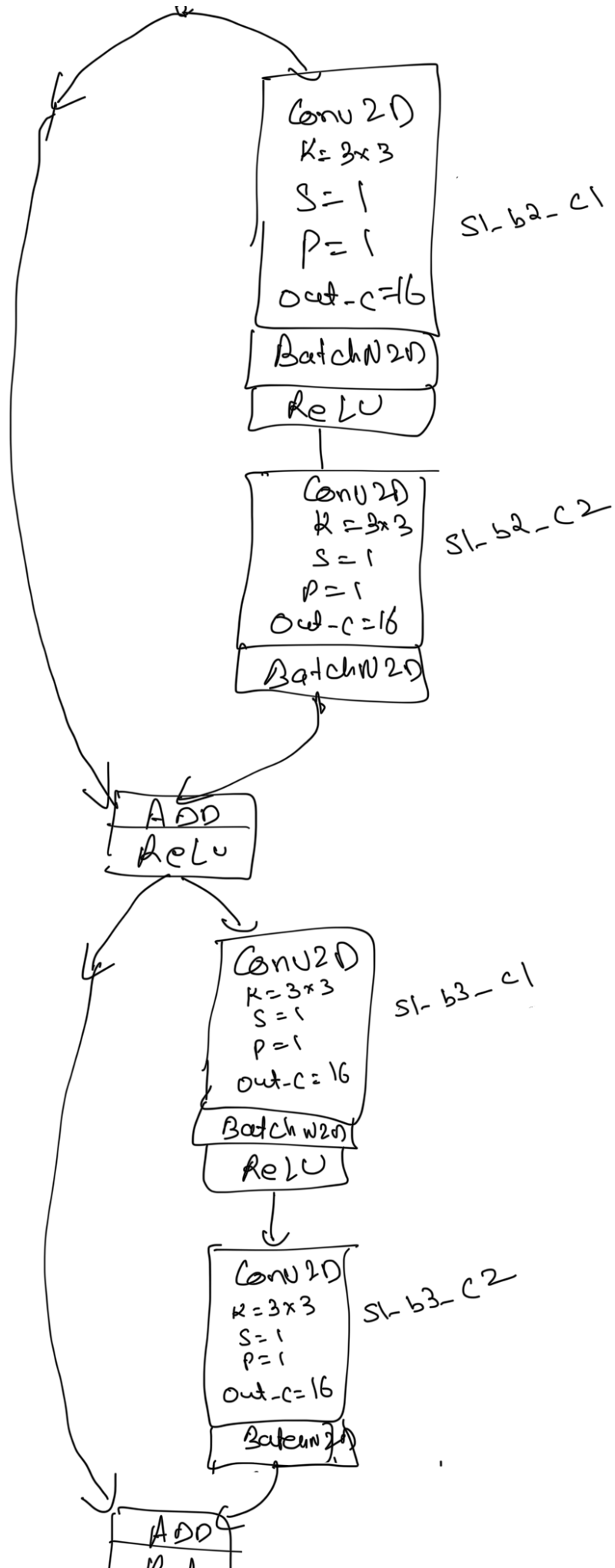


S1-b1-c2

32x32x16



Every
Stage
has
3 blocks

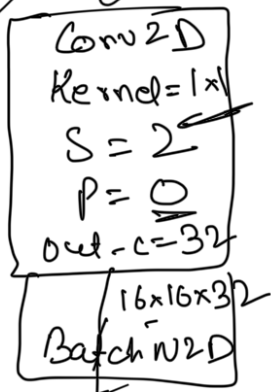
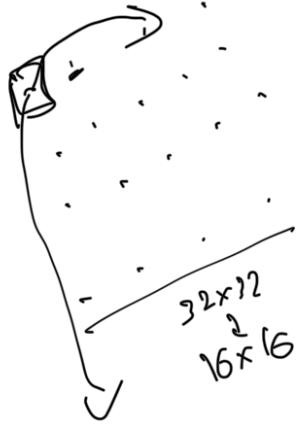


ReLU

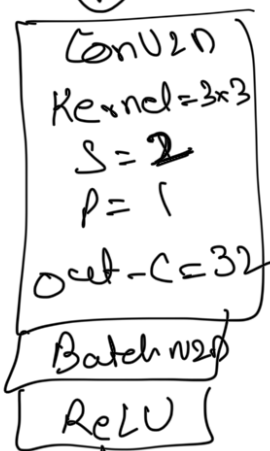
$32 \times 32 \times 16$

start with stage 2

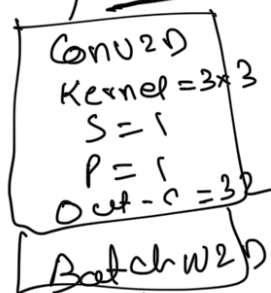
Increasing channels and downsampling image



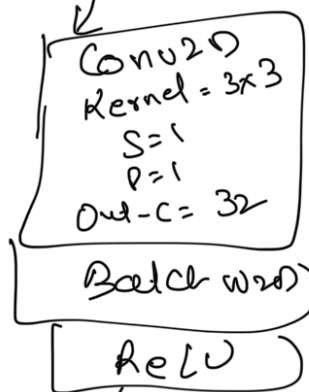
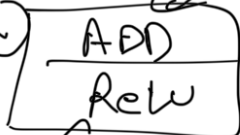
S2-skip



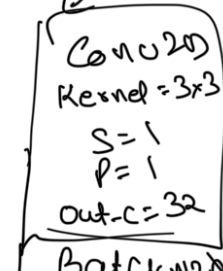
S2-b1-c1



S2-b1-c2



S2-b2-c1



S2-b2-c2

